

OuviSearch: Triagem Semântica de Manifestações Cidadãs

Representações Vetoriais, Busca Semântica, Detecção de Duplicatas e Chunking
Relatório do Desafio Prático - NLP Aplicado - UNIPÊ, Centro Universitário de João Pessoa
Disciplina: Tendências em Ciência da Computação
Professor: Me. Ricardo Roberto de Lima
Autor: Pedro Nícollas Pereira Leon Lopes

1. Contexto e Problema

A Ouvidoria Geral de um município fictício recebe milhares de manifestações por mês via formulário online, hoje triadas por busca de palavras-chave. Esse método tem três falhas conhecidas: (i) não detecta duplicatas semânticas escritas com palavras diferentes (ex.: "buraco na via" e "asfalto esburacado"); (ii) não agrupa manifestações do mesmo tema quando o vocabulário não coincide; (iii) perde contexto em textos longos indexados como bloco único. O OuviSearch é um protótipo de triagem semântica que ataca essas três falhas com representações vetoriais densas, detecção automática de duplicatas por similaridade de cosseno e chunking de textos longos.

2. Arquitetura da Solução

A solução foi dividida em quatro entregas, cada uma em seu próprio arquivo, todas consumindo a mesma base de dados (data/manifestacoes.json) e a mesma função de geração de embeddings com fallback documentado (ver seção 3).

Entrega	Arquivo	O que faz
1 - Comparação de representações	notebooks/analise_comparativa.ipynb	Gera BoW, TF-IDF e Embeddings para o corpus e compara a similaridade de cosseno em 3 pares de manifestações definidos no enunciado, discutindo as limitações de cada representação.
2 - Detecção de duplicatas	notebooks/deteccao_duplicatas.ipynb	Implementa detectar_duplicatas(textos, limiar), gera o heatmap de similaridade completo, calibra o limiar e analisa falsos positivos/negativos contra as duplicatas reais do dataset.
3 - Chunking	notebooks/chunking_manifestacoes.ipynb	Divide as 5 manifestações mais longas com RecursiveCharacterTextSplitter em duas configurações de (chunk_size, chunk_overlap) e mede a coesão semântica entre chunks vizinhos.
4 - App Streamlit	app_ouvidoria.py	Interface com 4 abas (Busca Semântica, Base Completa, Espaço Vetorial, Chunking), sidebar com seleção de modelo de embedding e top-k configurável.

3. Decisões de Projeto

3.1 Corpus sintético localizado, com ground truth conhecido

Como o enunciado não fornecia o arquivo manifestacoes.json, ele foi construído do zero com 40 manifestações (8 por categoria oficial), ambientadas em bairros e avenidas reais de João Pessoa e Campina Grande (PB), respeitando as exigências explícitas: os pares M003/M017 e M008/M022 (duplicatas) e M008/M031 (controle não relacionado) citados no enunciado, mais um terceiro par de duplicata (M014/M029), totalizando 3 pares (15% do corpus) e 5 manifestações longas (>500 caracteres, uma por categoria) para os exercícios de chunking. Conhecer o ground truth de antemão permitiu medir precisão e revocação reais na Entrega 2, em vez de apenas inspecionar resultados visualmente.

3.2 Modelo de embeddings e fallback documentado

Todos os resultados abaixo usam o modelo real paraphrase-multilingual-MiniLM-L12-v2 (sentence-transformers, 384 dimensões). Por robustez, a função de embeddings mantém um fallback: se a biblioteca ou o download do modelo não estiverem disponíveis, ela cai automaticamente em uma simulação com TF-IDF + SVD (LSA), o que mantém notebooks e app executáveis em qualquer ambiente. Essa simulação foi usada nas primeiras versões do projeto e serve de contraponto na seção 4.1.

4. Principais Resultados

4.1 Comparação de representações (Entrega 1)

Par	BoW	TF-IDF	Embeddings
M003 x M017 (duplicata)	0.4454	0.1558	0.5926
M008 x M022 (duplicata)	0.4170	0.3087	0.7547
M008 x M031 (controle)	0.3485	0.2072	0.2164

Só os embeddings separam duplicatas do par de controle: 0.59 e 0.75 contra 0.22. O BoW mal distingue M008/M022 (0.42) do controle (0.35), pois palavras funcionais compartilhadas inflam a similaridade de qualquer par. O TF-IDF corrige esse ruído, mas exige sobreposição literal de palavras: em M003/M017 ("buraco" vs. "esburacado", sem palavra de conteúdo em comum) ele dá 0.16 à duplicata real, MENOS que ao controle (0.21), invertendo o ranking esperado. O modelo pré-treinado reconhece a relação (0.59, quase o triplo do controle) por trazer conhecimento semântico externo ao

corpus. Como contraprova, o fallback TF-IDF + SVD (que só aprende com os 40 documentos) deu 0.16, 0.56 e 0.37 aos mesmos pares: a duplicata M003/M017 voltou a ficar abaixo do controle.

4.2 Detecção de duplicatas (Entrega 2)

Com o limiar do enunciado (0.85) nenhuma duplicata é encontrada: o maior score do corpus é 0.79. O limiar 0.75 é o mais alto que captura 2 das 3 duplicatas reais; abaixo dele os falsos positivos explodem (0.70 marca 12 pares, 0.65 marca 26). Com 0.75 são marcados 7 pares: precisão = 0.29 e revocação = 0.67. Acertam M014/M029 (2º lugar no ranking de 780 pares) e M008/M022 (5º); M003/M017 é falso negativo, na 43ª posição. Os 5 falsos positivos são, em quatro casos, pares do mesmo tema com problemas distintos (ex.: duas escolas diferentes sem merenda, score 0.79, maior que o de qualquer duplicata real) e, em um, um par de relatos longos de categorias diferentes. O modelo mede proximidade de assunto, não identidade de denúncia.

4.3 Chunking (Entrega 3)

Comparando `chunk_size=150` com `overlap=0` e `overlap=50` sobre as 5 manifestações mais longas, a coesão semântica média entre chunks consecutivos da mesma manifestação sobe de 0.2601 para 0.4142 (cerca de 1,6x). Sem `overlap`, frases são cortadas ao meio (ex.: em M004, "formais da associação de moradores." fica isolado do restante da frase); com `overlap`, o fim de um chunk se repete no início do próximo. O ganho é modesto porque mesmo sem `overlap` os chunks de um relato já compartilham tema e localização. Quanto ao espaço vetorial, chunks da mesma manifestação têm similaridade média 0.32 contra 0.19 entre manifestações diferentes, e em 21 de 34 chunks (62%) o vizinho mais próximo é da mesma manifestação: agrupamento real, porém parcial.

5. Dificuldades e Aprendizados

A maior dificuldade foi desenhar um corpus sintético pequeno (40 documentos) que respeitasse as exigências literais do enunciado (pares específicos, percentual de duplicatas, textos longos) e soasse localizado (bairros e avenidas reais de João Pessoa e Campina Grande). Os principais aprendizados vieram dos resultados, nem sempre os esperados. Primeiro, o caso M003/M017 mostra por que embeddings pré-treinados existem: BoW, TF-IDF e até uma LSA sobre o próprio corpus não reconhecem que "buraco" e "esburacado" descrevem o mesmo problema. Segundo, ter o modelo certo não resolve tudo: na detecção de duplicatas, nenhum limiar único separa bem duplicatas de reclamações apenas do mesmo tema, e o 0.85 sugerido não se aplica a este modelo e corpus (o teto é 0.79), pois a escala dos scores depende do modelo. Terceiro, medir vale mais que afirmar: ao quantificar o agrupamento de chunks (62% de vizinhos da mesma manifestação) e o efeito do `overlap` (1,6x, e não a diferença dramática que o modo de simulação sugeria), corrigimos expectativas iniciais com números.

6. Desenvolvimento e Divisão de Tarefas

Todas as etapas foram desenvolvidas individualmente por Pedro Nícollas Pereira Leon Lopes: construção do corpus sintético, implementação das quatro entregas (comparação de representações, detecção de duplicatas, chunking e app Streamlit) e redação deste relatório. A ferramenta de assistência por IA Claude Code (Anthropic) foi utilizada em partes pontuais, como auxílio na geração de conteúdo do corpus sintético e na revisão textual do relatório, cujo conteúdo foi verificado e validado integralmente pelo autor.

7. Conclusão

O protótipo demonstra, com números reproduzíveis, os problemas que motivam a triagem semântica: representações esparsas falham em duplicatas sem sobreposição lexical (o TF-IDF chega a inverter o ranking), embeddings pré-treinados reconhecem a paráfrase, mas a similaridade de cosseno sozinha mede proximidade de assunto e não identidade de denúncia. Na prática, ela serve como triagem: reduz 780 pares a cerca de 7 candidatos para revisão humana, ainda que com revocação de 2 em 3 duplicatas e sem substituir a decisão do revisor. Como próximos passos, combinar o score com sinais estruturados (mesmo logradouro, mesma categoria, datas próximas) ou reavaliar os candidatos com um cross-encoder tende a elevar a precisão da detecção de duplicatas.